UNIVERSITY OF AMSTERDAM

UvA-DARE (Digital Academic Repository)

Centrally Coordinated Multi-Agent Reinforcement Learning for Power Grid Topology Control

de Mol, Barbera; Barbieri, Davide; Viebahn, Jan; Grossi, Davide

DOI
10.1145/3679240.3734602

Publication date
2025

Document Version
Final published version

Published in
E-ENERGY '25

License
CC BY

Link to publication

Citation for published version (APA):
de Mol, B., Barbieri, D., Viebahn, J., & Grossi, D. (2025). Centrally Coordinated Multi-Agent Reinforcement Learning for Power Grid Topology Control. In *E-ENERGY '25: Proceedings of the 2025 16th ACM International Conference on Future and Sustainable Energy Systems : June 17-20, 2015, Rotterdam, Netherlands* (pp. 460-475). Association for Computing Machinery. https://doi.org/10.1145/3679240.3734602

General rights
It is not permitted to download or to forward/distribute the text or part of it without the consent of the author(s) and/or copyright holder(s), other than for strictly personal, individual use, unless the work is under an open content license (like Creative Commons).

Disclaimer/Complaints regulations
If you believe that digital publication of certain material infringes any of your rights or (privacy) interests, please let the Library know, stating your reasons. In case of a legitimate complaint, the Library will make the material inaccessible and/or remove it from the website. Please Ask the Library: https://uba.uva.nl/en/contact, or a letter to: Library of the University of Amsterdam, Secretariat, P.O. Box 19185, 1000 GD Amsterdam, The Netherlands. You will be contacted as soon as possible.

UvA-DARE is a service provided by the library of the University of Amsterdam (https://dare.uva.nl)

Download date:20 Oct 2026

Centrally Coordinated Multi-Agent Reinforcement Learning for Power Grid Topology Control

Barbera de Mol
EDMij B.V.
Ouderkerk aan de Amstel, Netherlands
barbera.de.mol@edmij.nl

Davide Barbieri
TenneT TSO
Arnhem, Netherlands
davide.barbieri@tennet.eu

Jan Viebahn
TenneT TSO
Arnhem, Netherlands
jan.viebahn@tennet.eu

Davide Grossi
University of Groningen
Groningen, Netherlands
University of Amsterdam
Amsterdam, Netherlands
d.grossi@rug.nl

Abstract

Power grid operation is becoming more complex due to the increase in generation of renewable energy. The recent series of Learning To Run a Power Network (L2RPN) competitions have encouraged the use of artificial agents to assist human dispatchers in operating power grids. However, the combinatorial nature of the action space poses a challenge to both conventional optimizers and learned controllers. Action space factorization, which breaks down decision-making into smaller sub-tasks, is one approach to tackle the curse of dimensionality. In this study, we propose a centrally coordinated multi-agent (CCMA) architecture for action space factorization. In this approach, regional agents propose actions and subsequently a coordinating agent selects the final action. We investigate several implementations of the CCMA architecture, and benchmark in different experimental settings against various L2RPN baseline approaches. The CCMA architecture exhibits higher sample efficiency and superior final performance than the baseline approaches. The results suggest high potential of the CCMA approach for further application in higher-dimensional L2RPN as well as real-world power grid settings.

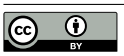
CCS Concepts

• **Computing methodologies** → **Multi-agent reinforcement learning**; Adversarial learning; *Multi-agent systems*; • **Hardware** → **Power networks**.

Keywords

Multi-Agent Reinforcment Learning (MARL), Hierarchical Reinforcement Learning (HRL), Consensus-Based Learning, Power Network Control (PNC)

ACM Reference Format:
Barbera de Mol, Davide Barbieri, Jan Viebahn, and Davide Grossi. 2025. Centrally Coordinated Multi-Agent Reinforcement Learning for Power Grid Topology Control. In *The 16th ACM International Conference on Future and Sustainable Energy Systems (E-ENERGY '25), June 17–20, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3679240.3734602



This work is licensed under a Creative Commons Attribution 4.0 International License.
E-ENERGY '25, Rotterdam, Netherlands
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1125-1/25/06
https://doi.org/10.1145/3679240.3734602

1 Introduction

The growing demand for electricity, driven by technological advancements and the shift towards electrified industries and transportation, highlights the need for resilient power grids. Managing these grids involves complex sequential decision-making across large state and action spaces, further complicated by aging infrastructure and increased reliance on unpredictable renewable energy sources [24, 30].

Traditional grid management methods, such as redispatching and building new infrastructure, are costly and dependent on external factors. However, topological remedial actions, like reconfiguring grid substations, offer a cost-effective alternative that remains underexplored [13, 39, 40]. As power grids expand, conventional computational methods struggle to provide optimal real-time control solutions, leading operators to rely on experience or predefined manuals. The combinatorial complexity of grid configurations necessitates more advanced approaches.

To support research in *power network control* (PNC), the Grid2Op framework was developed [6], enabling simulation of realistic grid scenarios as a Markov Decision Process (MDP) [36]. This framing makes the problem suitable for deep Reinforcement Learning (RL), which has been successful in various domains [9, 12, 17, 33].

However, deep RL faces scaling issues with large networks due to the combinatorial explosion of state and action spaces. The curse of dimensionality remains a major bottleneck in RL tasks. Specifically, the sample complexity grows exponentially with the dimensionality of the state-action space of the environment, posing challenges for large-scale applications. To address these challenges, Hierarchical Reinforcement Learning (HRL) and Multi-Agent Reinforcement Learning (MARL) offer promising solutions by decomposing complex tasks into simpler sub-tasks and distributing decision-making across multiple agents [3, 21]. This idea is captured in the factored MDP framework [29], where the original MDP is decomposed into a set of smaller and independently evolving MDPs. In this case, the total sample complexity is determined by the sum of the sizes of

the state-action spaces for each individual MDP, rather than their product. As a result, the problem size no longer scales exponentially with the dimension of the problem.

This study integrates the strengths of MARL and HRL to develop a centrally coordinated multi-agent (CCMA) architecture. The design incorporates short-term action suggestions alongside a coordinating agent capable of managing multi-step action sequences. Different algorithms for short-term action suggestions and coordinating agents are compared, ranging from greedy and rule-based strategies to fully RL-based architectures.

The main contributions can be summarized as follows:

- We introduce a novel CCMA architecture featuring a coordinator that determines the next action by leveraging regional action suggestions from individual agents alongside global state information. This design enables action space factorization while maintaining centralized coordination.
- We describe several variants of the CCMA architecture, incorporating rule-based, greedy, and RL-based modules. These architectures range from fully deterministic approaches (rule-based and greedy) to fully learned systems (entirely RL-based) and hybrid designs that combine both strategies.
- We evaluate the proposed architecture against baseline models [6, 7], demonstrating that some of its variants surpass all baselines in both sample efficiency and overall performance, hence effectively factor the MDP.
- Finally, we show that rule-based coordinators are effective in the smallest 5-bus network. However, when tested with larger networks, these architectures no longer match the performance of an RL-based coordinator.

The paper is organized as follows: Section 2 covers the background and related work. Section 3 defines the control framework and Section 4 describes the power grid environment. The central Section 5 introduces the CCMA architecture. Finally, Section 6 presents the experiments and results, followed by a discussion and future work in Section 7.

2 Background and Related Work

This section deals with the modeling of the PNC problem (Section 2.1), applications of RL in this domain (Section 2.2), and recent HRL and MARL implementations that focus on improving sample efficiency and overcoming the curse of dimensionality (Section 2.3).

2.1 Power Network Control

Secure operation of power networks is required both in normal operating states as well as in contingency states (i.e., after the loss of any single element on the network). That is, the following requirements must be met: (i) In the normal operating state, the power flows on equipment, voltage and frequency are within pre-defined limits in real-time; (ii) In the contingency state the power flows on equipment, voltage and frequency are within pre-defined limits. Loss of elements can be anticipated (scheduled outages of equipment) or unanticipated (faults for lightning, wind, spontaneous equipment failure). Cascading failures must be avoided at all times to prevent blackouts (corresponding to the game over state in the RL challenge).

Importantly, each of the control actions performed by power system operators usually not only affects the current state of the power system but also the future state and availability of future control actions, that is, short-term actions can have long-term consequences. As a result, the decision problem of power system operators is typically a sequential decision-making problem in a combinatorial action space in which the current decision can affect all future decisions. Moreover, due to possible nondeterministic changes of the power system state (e.g., due to unplanned outages or the intermittent behaviour of renewable energy sources) and different sources of error (e.g., measurement errors, state estimation errors, flawed judgement) the operators need to handle uncertainty in their decisions. Finally, operational decisions must often be made quickly, under hard time constraints [40].

2.2 Deep RL for Power Network Control

Deep RL has demonstrated significant potential in PNC, enabling robust and adaptable behavior over extended time horizons [40]. This capability surpasses the limitations of expert systems [28] and optimization methods, which are constrained by computation time [11, 20, 32].

In L2RPN competitions, successful solutions typically combine expert rules, RL agents, and brute force simulations for action validation. Notably, RL agents based on Proximal Policy Optimization (PPO) [34], with reduced action spaces, have performed well. This approach was used in top-ranking competition entries (e.g. [27], further explored in studies [2, 15, 16, 25]).

Several commonalities emerge among top-ranking approaches. Most successful methods implement intervention only during hazardous situations [2, 8, 14, 16, 22, 25, 41]. In addition, many such approaches integrate learned modules with simulation-based action evaluation. For instance, this strategy was key to the success of the winning teams in both 2019 [14] and 2021 [31]. Notably, the second-place competitor in 2021 employed an advanced expert system, underscoring the value of incorporating domain knowledge from power systems [26].

2.3 Curse of Dimensionality

The combinatorial complexity of topological actions in PNC poses a challenge for deep RL by hindering complete and consistent convergence and exploration of action and state spaces. HRL simplifies learning by breaking tasks into subtasks, enabling more focused and sample efficient learning per task as well as increased interpretability. MARL distributes decision-making across agents, effectively factorizing the MDP's action space.

2.3.1 Hierarchical Reinforcement Learning. In the L2RPN WCCI 2020 competition, the winning agent for a 36-bus network used a two-tiered framework: a high-level policy proposed a goal topology, while a low-level policy determined the sequence of individual topological actions [41].

Later, a three-level HRL framework was introduced [25], where the top level activates the agent in hazardous situations, the intermediate level selects a substation using RL, and the lowest level identifies a configuration. They tested greedy and RL-based approaches for the lowest level, and compared PPO and Soft Actor Critic (SAC) [10] for the RL-based modules. They found PPO to

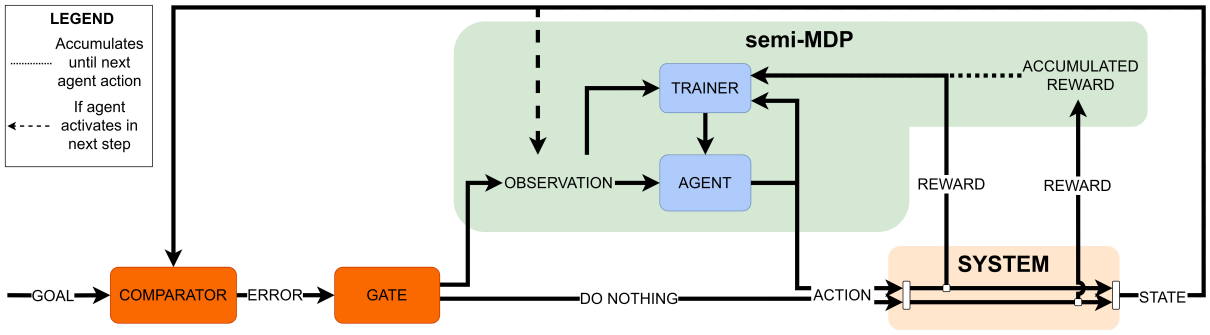


Figure 1: Diagram of the Feedback Control Framework used in this research. The goal to be achieved (and maintained) in the system is an input to the entire framework. The comparator determines the difference between the current state of the system and the desired one. There are then two possible paths for the control flow. If a goal state is reached then no action is performed and the reward is accumulated. If the system is not in the goal state then an agent is called. Periodically this agent can be updated by a trainer. The dashed line represents the only link between the two loops, as the trainer will perceive the accumulated rewards of consecutive time-steps where the *do-nothing* was used for control

have faster convergence, smaller variance, and higher expected rewards.

2.3.2 Multi-Agent Reinforcement Learning. The action space in PNC can be factorized into exclusive subsets, one for each substation, enabling a natural MARL framework in a fully cooperative setting where all agents optimize the same collective objective. Following this approach, van der Sar et al. [38] extended Manczak et al. [25] by introducing multiple agents at the lowest level, with each substation controlled by an agent and a heuristic-based intermediate level. They trained agents using independent PPO (IPPO) and dependent PPO (DPPO). On a 5-bus network, both achieved optimal scores, though single-agent PPO converged faster. However, they anticipate that MARL's advantages will become more pronounced when scaled up to larger networks.

In other domains, Yu et al. [42] showed strong PPO-based multi-agent performance in testbeds, recommending best practices like value normalization, integrating global and local information, and limiting sample re-use to avoid training instability from MARL non-stationarity.

3 Control framework

Figure 1 shows a schematic of the feedback control (FC) framework employed in this study. The *goal* represents a target state that is to be maintained within the controlled *system* (light orange box). The *comparator* (left orange box) computes the *error* between the current and the goal state of the system. The controller produces an *action* to be performed in the system in order to get closer to or maintain the goal state of the system. The system produces a *reward* signal and a new *state* once the action is performed.

As shown in Fig. 1, our controller has two modes of operation. After the error is computed, a *gate* (right orange box) determines which mode to use. If the goal state is achieved (error = 0) then the gate selects the lower mode of operation in which a constant action ("do nothing") is performed in the system. If the goal state is not achieved then an *agent* (lower blue box) is used to compute an action. The state of the system (and possibly the error) is used to create the input *observation* for the agent.

The agent can either be rule-based or trained. In case of a trained agent, the agent mode of operation stores past experiences of its interaction with the system for training the agent. The controller can switch between modes of operation multiple times. Switching from the agent to the do-nothing operation may be the result of the agent's action. The system may also remain in the do-nothing mode of operation for multiple timesteps. In this case, the resulting rewards are accumulated. Once the agent mode of operation is used again, the *trainer* (upper blue box) will receive these accumulated rewards. This allows the agent to receive feedback for the effect that its action has on the gate. Effectively, the agent mode of operation is thus operating as a semi-MDP (sMDP) [1, 37]. The trainer component periodically uses collected experiences to update the parameters of the agent. We note that the trainer does not update the agent every time-stamp. Instead, this happens only periodically.

4 Power System Environment

First, in Section 4.1, we describe the power grids used for experimental analysis, which is the *system* in Fig. 1. Subsequently, in Section 4.2, we describe the corresponding *gate*, *state*, *reward*, and *actions*.

4.1 Experimental Setup

We consider two simulated power grids: the *IEEE case 5* and *IEEE case 14* networks. In the Grid2Op simulator [6], the power grid environments are called *rte_case5_example* and *l2rpn_case14_sandbox* (see the Grid2Op documentation for visualizations of these power grids). The 5-bus network comprises twenty scenarios, each spanning 2016 timesteps. The 14-bus network contains 1004 scenarios, each unfolding over 8064 timesteps. Each timestep, representing a five-minute interval, incorporates unknown changes in loads and production.

Moreover, unplanned outages can be included. These outages involve temporarily disconnecting a powerline for a specified duration. These unplanned outages are initiated by an opponent, adding stochasticity and further increasing task complexity. This opponent introduces realistic challenges faced by power grid operators, ensuring the system's stability is preserved even during unforeseen contingencies. The opponent is implemented as Grid2Op's *RandomLineOpponent* [5]. It is designed to randomly disconnect a line from a predefined set of targets, with a cooldown period implemented to delay consecutive attacks, identical to Manczak et al. [25]. After each attack, the disconnected line is automatically restored. Table 1 outlines the parameters of these attacks.

Table 1: The used parameters for the opponent in Grid2Op, namely the attackable lines ($l_{origin-extremity}$), the attack duration (in timesteps), and frequency (in timesteps).

	5-bus	14-bus
Attackable lines	l_{0-2}, l_{0-4}	$l_{3-4}, l_{3-6}, l_{3-8}, l_{11-12}$
Attack duration (ts)	48	48
Attack cooldown (ts)	144	144

4.1.1 Incorporated Domain Knowledge. Several domain-inspired features are incorporated in all baselines and architectures to boost performance. First, automatic powerline reconnection is implemented. Powerlines disconnected by overcurrent events face a ten-timestep cooldown before reconnection. Since agents are limited to topological actions, disconnected lines are automatically restored to their previous state one timestep after disconnection. This eliminates the need for the agent to handle reconnections, allowing it to focus exclusively on optimizing grid configurations.

Second, grids tend to be more stable during nighttime. This is due to lower demand (and thus strain) on the grid. This domain knowledge can be exploited to automatically improve the system's performance by returning the grid back to its original grid state where all powerlines within each substation are interconnected [16]. In this research, this is implemented to happen between 3AM and 6AM. Before this reversion is performed, the agent simulates whether this action would not likely lead to a game-over state, only performing the action if all estimated relative powerline thermal loads remain below 90% of their maximum capacity.

Lastly, it is worth noting that the grid is typically most stable during nighttime. Normally, agents navigate scenarios ranging from 2016 timesteps (for the 5-bus network) to 8064 timesteps (for the 14-bus), equivalent to seven to 28 days. If an agent fails during training, the scenario terminates. To expose agents to diverse circumstances, scenarios can be subdivided into smaller segments, increasing the likelihood that the agent encounters later timesteps in longer scenarios, potentially speeding up convergence. We therefore split the scenarios into chunks of two days, starting during nighttime.

4.2 Power System Feedback Control Elements

This subsection explains how each component of the FC framework presented in Sec. 3 is realized in a power system setting.

Table 2: A comparison between the 5-bus and 14-bus networks. Only topological actions are considered and the do-nothing action is excluded.

	symmetry-filtered		N-1 secure	
	# actions	# topologies	# actions	# topologies
5-bus	58	31320	24	364
14-bus	178	3.9e+11	73	3.3e+5

Goal. Obviously, the general goal is to maintain a grid state without any overloaded lines. In this study, we adopt the approach employed in most L2RPN studies. Specifically, the grid state is characterized by the loading of individual power lines, represented as a vector $\rho = (\rho_1, \rho_2, \ldots, \rho_n)$ where each element ρ_i denotes the loading of a specific line in the power grid. The goal state is defined by a threshold $\tilde{\rho}$, a single scalar value representing the allowable maximum line loading. The goal is to maintain $\max(\rho) < \tilde{\rho}$, where $\max(\rho)$ refers to the maximum loading among all the lines in the grid. In this study, we set $\tilde{\rho} = 0.95$.

Gate. We use a common rule-based gate. If the maximum line loading is lower than the threshold $\tilde{\rho}$ then the do-nothing mode of operation is used. If the maximum line loading surpasses the threshold $\tilde{\rho}$ then the agent mode of operation is used.

State. The state of the system consists of (i) the current grid topology, (ii) the current load ρ_l on each powerline l, measured as the fraction of the capacity of the powerline, (iii) the active power flow at each end of each powerline, (iv) the number of overflow timesteps for each powerline, and (v) the current power production and consumption of generators and loads.

In Grid2Op, if a blackout occurs, it results in a game-over state. A blackout happens when the power grid fails to meet the requirements of the generators and loads due to issues like line disconnections, overloading, or other critical failures. Once a blackout is detected, the simulation ends immediately, as the grid's stability can no longer be maintained.

Reward. We employ the scaled L2RPN reward [6, 25], which calculates the squared margin of the current flow relative to the current limit for each powerline. The larger the margin, meaning the further the current flow is from the limit, the higher the reward, encouraging states where powerlines are not operating near their maximum capacity.

Actions. In this study, we introduce a new action space designed to ensure structural N-1 security. Structural N-1 security means that the grid remains operational and avoids an automatic blackout in the event of a single contingency, such as an unplanned powerline outage. We compare the new action space with the default action space which only excludes symmetrical counterparts from all possible substation configurations.

Table 2 shows the size of each action space.

The size of the symmetry-filtered action space can be computed using two key terms. The first term, $\alpha(n) = 2^{n-1}$, represents the total number of possible configurations for a substation with two busbars and n elements connected to it, excluding symmetrical

configurations (a factor of $\frac{1}{2}$). However, this term does not account for the constraint that no loads or generators (collectively referred to as injections) should be isolated.

The second term, $\gamma(n') = 2^{n'} - 1$, accounts for all configurations where only injections and no powerlines are connected to a busbar. Here, n' is the number of injections connected to the substation. The $2^{n'}$ represents all possible combinations of injections in a configuration where all powerlines are connected to a single busbar, and the -1 excludes the configuration where all injections are also connected to this single busbar.

Using these terms, the size of the symmetry-filtered action space is calculated as:

$$\tau_{sym} = \alpha(n) - \gamma(n') \ . \tag{1}$$

By always maintaining zero or at least two powerlines connected to each busbar, the N-1 secure action space ensures that the failure of a single powerline does not result in an immediate game-over state due to the loss of connection to its injections. The size of the structurally N-1 secure action space is:

$$\tau_{N-1} = \alpha(n) - \gamma(n') - \epsilon(n', n'') \ , \tag{2}$$

with:

$$\epsilon(n', n'') = 2^{n'} \cdot (n'' - \delta_{n'',2} - \delta_{n'',1}) \ , \tag{3}$$

where n'' is the number of powerlines, and $\epsilon(n', n'')$ removes all configurations where only one powerline is connected to a busbar. The Kronecker delta, $\delta_{n'',k}$, is defined as:

$$\delta_{n'',k} = \begin{cases} 1 & \text{if } n'' = k, \\ 0 & \text{otherwise.} \end{cases}$$

This means that $\delta_{n'',2}$ is one when exactly two powerlines are present ($n'' = 2$) and zero otherwise, while $\delta_{n'',1}$ is one when only one powerline exists ($n'' = 1$) and zero otherwise. These terms ensure that configurations with invalid powerline setups, such as only one powerline connected or configurations invalid for all possible injection setups, are excluded. In practice, such cases are always kept in the base topology where all elements are connected.

Combined, τ_{N-1} excludes all configurations that connect any injection to only a single powerline, ensuring structural N-1 security. This equation is valid as long as at least one powerline exists.

5 Power system agents

In this section, we present the agent configurations used in the second mode of operation of the FC framework (see Sec. 3, Fig. 1). The greedy agent and the single RL agent (Section 5.1) approach the sMDP without decomposition. In contrast, this study introduces a multi-agent architecture that divides the sMDP into a smaller set of independent tasks (see Sec. 5.2). The two common baselines serve as a reference to evaluate the impact of decomposing the MDP.

5.1 Greedy Agent & Single RL Agent

The greedy-agent baseline is a fully rule-based agent. A greedy agent simulates the effect of every action in the action space for one timestep in the future. Then, the agent selects the action that maximizes a given KPI when simulated. In our case, the KPI is the maximum loading of the grid. See de Jong et al. [4] for a pseudo-code of the greedy agent.

The single RL agent uses a feed-forward neural network to implement a policy for choosing actions. It also chooses actions from the entire action space. A state-of-the-art RL algorithm is used to train the policy, namely, the Proximal Policy Optimization (PPO) algorithm [34].

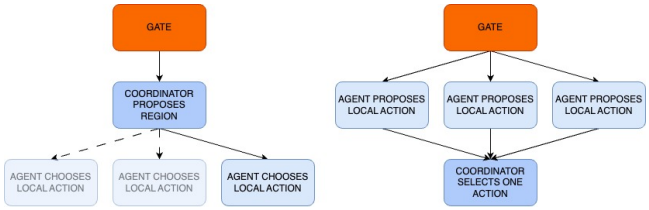


Figure 2: Left: Hierarchy proposed by Manczak et al. [25], van der Sar et al. [38]. First a region (e.g., a substation) is selected, and then a topological configuration for that region (and that region only) is selected. Right: Hierarchy proposed in this study. For each region a topological reconfiguration is proposed concurrently, and a coordinator then selects the best proposed action (e.g., a substation configuration).

5.2 Multi-Agent Architecture

We divide the overall agent task into two levels of abstraction, following the approaches outlined in Manczak et al. [25], van der Sar et al. [38]. One level consists of deciding the topological reconfiguration of individual regions (e.g. substations). The other level consists of selecting which region should be reconfigured. The agent selecting the region is called the *coordinator*. The reconfiguration of regions may be decided by multiple agents, called *regional agents*, each proposing a topological reconfiguration of its respective region. In our architecture the topological reconfiguration of each individual region is decided first, and the region to be reconfigured is selected second. Figure 2 illustrates on the left the high-level hierarchy used in Manczak et al. [25], van der Sar et al. [38] where the regional agents are called after the coordinator, and on the right the decision making order proposed in this study.

Our architecture allows the coordinator to take into account the proposed regional reconfiguration in its decision making process. We believe that this allows for a more clean separation between the two levels of abstraction. In Manczak et al. [25], van der Sar et al. [38], the coordinator may need to anticipate implicitly what reconfiguration will be proposed for the region it is selecting. This may result in the coordinator having to solve both tasks as a single agent. In other words, we believe that the regional topology selection task needs to be solved (either implicitly or explicitly) before being able to determine which region to select.

Figure 3 provides a more detailed description of our architecture. At the top level, there is always the FC framework's gate. When switching to the agent mode of operation, the regional agents are activated. Each of them receives the current observation and subsequently proposes a topological reconfiguration for its region[1]. All of the proposed regional reconfigurations are passed forward to the

[1]A regional agents may also still propose a *do-nothing* action for its respective region.

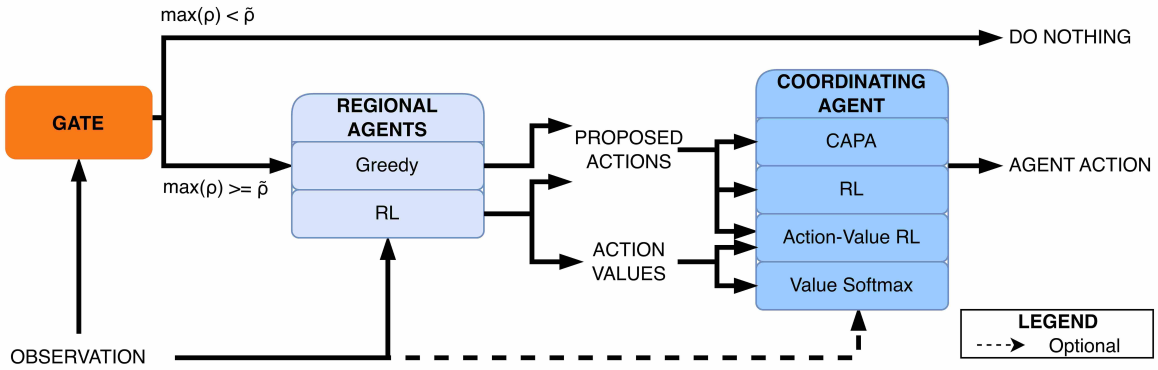


Figure 3: An overview of all possible multi-agent architectures. First the maximum line loading in the current observation OBS is compared to a threshold by the gate to determine whether to not act (upper path) or to try to reconfigure the topology of the grid (lower path). In the latter case, first all the REGIONAL AGENTS propose an action to reconfigure their respective region (or do nothing). This list of action is finally passed to the COORDINATING AGENT which select one of the regions (i.e. one of the actions proposed by any of the regional agents). The final result is always an action in the topological reconfiguration space.

coordinator. The latter may take into account all of the proposed actions and the current observation. If available, the coordinator also receives the action-value estimates, which represent the network's internal state and quantify the predicted utility or confidence in the effectiveness of each action.

Each regional agent only experiences rewards (and thus transitions) if it is selected by the coordinator. When the gate selects consecutive do-nothing actions, the accumulated reward is assigned to the last regional agent that was selected by the coordinator.

Figure 3 also shows the different implementations of the regional agents and the coordinator considered in this study. In this paper, each regional agent is either a RL agent trained using the PPO algorithm [34], or a greedy rule-based method (see Sec. 5.1). For the RL agent, the action-value of the proposed action (as evaluated by the critic) may also be passed forward.

The coordinator also has multiple implementations. When the coordinator is trained via RL and the action-value of the regional agent is included then the coordinator is referred to as *Action-Value RL* agent. If the action-value is not passed forward, the coordinator is simply referred to as *RL* agent. We also experiment with two rule-based implementations of the coordinator. The *CAPA heuristics* coordinator uses the *CAPA* algorithm [41]. Algorithm 1 shows our version of the CAPA coordinator, which looks at the actions proposed by the regional agents only to exclude those who proposed do-nothing actions. Moreover, this agent does have memory for the purpose of exhausting a priority list between regions before generating a new one. The other rule-based coordinator selects a regional agent based solely on the action-value of their actions[2]. More precisely, the *Value Softmax* coordinator samples a regional agent's action with a probability proportional to their action-value in order to balance exploitation of the action-values once the critics are sufficiently trained. This also allows for exploration thus avoiding biasing training data only towards specific regional agents.

[2]Note that these action-values are proposed by independent critics, one for each regional agent.

Algorithm 1 Ordered CAPA Algorithm

Given observation *obs*, activation threshold *max_rho* and an initial priority list of actions P that acts as the state of the algorithm
$current_rho \leftarrow max(rho)$ in *obs*
if $P = None$ or P is empty **then**
  Generate priority list P using CAPA policy
end if
for each action A **do**
  if $A \neq do_nothing$ **then**
    Remove A from P
    return A and P
  end if
end for
$P \leftarrow None$
return *do_nothing* and P

We note that if the *coordinating agent* uses a rule-based method that takes into account only the current observation to select a region (to then simply return whatever action that region's agent has proposed) then we effectively also obtain the architecture shown in Fig. 2 **(left)**. This is because, in such a case, it is irrelevant whether or not the regional agents propose the actions before or after the coordinating agent.

Table 4 in Appendix A highlights the key differences between the proposed CCMA architecture and other state-of-the-art architectures in power grid topology control.

5.3 Overview of power system agents

We summarize all considered approaches using the following naming convention: The full multi-agent system is indicated by [name regional agent]-[name coordinating agent], with the names given as in Table 3 and Fig. 3.

Table 3: The input for each module in the agent architecture.

Agent Name	Input
Regional Agents	
Greedy	Observation
RL	Observation
Coordinating Agents	
CAPA	Regional Actions & Observation
RL	Regional Actions & Observation
Action-Value RL	Regional Action-Value Pairs & Observation
Value Softmax	Regional Action-Value Pairs

5.3.1 Baselines. To evaluate the proposed approach, results are compared with several baselines that should at least be met in terms of performance:

- **Do-Nothing Agent**: This baseline shows the number of timesteps survived when no action is taken at any timestep.
- **Single RL Agent**: This baseline uses a single PPO agent responsible for configuring the entire network. The agent receives network observations as input and outputs a single action.
- **RL-CAPA**: This architecture combines a rule-based CAPA coordinator with RL-based regional agents. The CAPA policy prioritizes substations at risk of immediate overloading, using the highest relative overload value for selection. This approach ensures that regional agents are trained in scenarios where their actions are likely to have a significant impact. This adapted CAPA policy based on van der Sar et al. [38] also considers past actions to avoid experience imbalance and repeated selection of the same substation. If a proposed action does not alter the grid topology, the next highest-priority substation proposes an action, as detailed in Alg. 1.

5.3.2 Benchmarks. Two benchmark agents are designed using action simulation and greedy strategies. Both architectures are computationally intensive, as they simulate all possible actions and select the optimal one in a greedy manner. However, they serve to illustrate the complexity within networks and scenarios, demonstrating the potential scope of learning.

- **Greedy Agent**: This baseline involves a greedy agent that makes decisions based on minimizing the maximum relative line loading of the network. It operates without hierarchical coordination and provides a benchmark of what a simple architecture with enough computational power can achieve.
- **Greedy-CAPA**: To test the rule-based potential of a hierarchical design, greedy regional agents are paired with the CAPA coordinator.

5.3.3 Multi-agent RL architectures. For the proposed CCMA architecture several implementations are explored:

- **Greedy-RL**: This setup features a hierarchical structure with an RL-based coordinator that receives proposals from regional agents to address congestion. Unlike a greedy agent, the RL coordinator benefits from implicit planning, similar to the *PPO Substation* implementation by Manczak et al. [25]. The key distinction is that Manczak et al. [25]'s implementation does not involve multiple regional agents or consider the greedy agent's planned actions.
- **RL-RL**: This architecture features both a learned coordinator and learned regional agents. The coordinator utilizes both global state observations and regional agent information, inspired by the approach suggested by Yu et al. [42]. This setup allows the system to learn multi-step actions but may introduce instability due to the interactions among multiple learned levels and agents.
- **RL-Action Value RL**: This approach extends the previous setup by incorporating the action-value function outputs from regional agents as additional input for the learned coordinator. This allows for a more refined decision-making process by integrating the value estimates of different actions, potentially enhancing the coordination and overall performance of the system.
- **RL-Value Softmax**: This architecture uses the action-value function outputs from regional agents. A coordinator employing *softmax* (see Equation 5 below) assigns probabilities to actions during training, promoting exploration and fair experience distribution. During evaluation, *argmax* is used to choose the optimal action based on the trained agents.

5.4 Implementation details

To improve learning, avoid overfitting, and ensure a fair comparison between architectures, several measures are implemented for each experiment, including train-validation-test splitting, scenario splitting, and identical hyperparameter tuning.

5.4.1 Trainer. For all experiments, PPO [34] is used for learning a policy. The policy, denoted by π, is parameterized using a neural network with learnable parameters θ. The action-value function, represented as $Q(s_t, a_t)$, and the state-value function $V(s_t)$, are represented by a neural network with parameters ϕ.

For both training and evaluation, PPO is employed as a stochastic policy, meaning actions are sampled from a probability distribution rather than being chosen deterministically. Common practices such as including Generalized Advantage Estimation (GAE) are adopted to reduce variance and improve training stability [42]. The PPO implementation by Liang et al. [18] is utilized in this research.

For the multi-agent implementation, it is common that one agent acquires significantly more experience than other agents. To counteract this, we adapted PPO to postpone training until all agents have reached the minimum batch size or until any agent has reached double the minimum batch size.

5.4.2 Machine Learning Practices. Hyperparameter tuning is critical in RL research and applications because hyperparameters greatly affect the performance, stability, and efficiency of RL algorithms. For each architecture and environment, two grid searches are conducted to tune the parameters using Ray Tune [19]. The first is a coarse grid search to identify suitable learning rates, batch sizes, minibatch sizes, and the number of iterations. The second grid

search is narrower, focusing on smaller variations in these parameters, as well as additional parameters such as the clipping range, value function coefficient range, and GAE lambda range. Appendix B details the full grid search settings for all experiments.

It should be noted that to validate the stability and reliability of the proposed method, all architectures are trained independently per network using ten different random seeds. The stability of the approach is illustrated explicitly in Figure 4, where the variability across multiple seeds is represented by the standard deviation, indicating the robustness and convergence behavior under diverse initialization conditions.

Additionally, a priority function is integrated into the selection of the next training scenario to accelerate learning [38, 41]. This function assigns a logit to each scenario based on its characteristics, as defined by Equation 4:

$$w_{prio} = 1 - \left(2 \cdot \sqrt{\frac{t_{survived}}{t_{max}}}\right) \tag{4}$$

where w_{prio} represents the logits, $t_{survived}$ is the number of timesteps the agent survived in the last iteration, and t_{max} is the maximum number of timesteps for that scenario. The logits are scaled between -1 and 1: fully completed scenarios are assigned a value of -1, scenarios that fail at the start are assigned a value of 1, and unseen scenarios are initialized with a logit of 2 to encourage exploring new scenarios first. These logits are then used in a softmax distribution (Eq. 5) to sample the next scenario:

$$\sigma(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}, \tag{5}$$

where $\sigma(x_i)$ represents the probability of selecting scenario i, and the denominator ensures the probabilities are normalized by summing the exponential values of all scenario logits.

To avoid overfitting and ensure a robust model, all scenarios are divided into a 70%/10%/20% train/validation/test split.

6 Results

Figure 4 shows the learning curves of the different agent architectures for the four different experimental setups, namely, the agent performances during training on all validation scenarios, averaged over all ten model seeds[3].

First, we note that all agent architectures considered in this study, except the do-nothing agent, achieve optimal performance in the 5-bus environment without opponent (upper-left panel in Fig. 4, first column in Table 9). Consequently, the 5-bus environment without opponent can be considered as a baseline (or code testing) environment which is used to demonstrate that the minimally expected learning behavior is exhibited. Hence, in the subsequent sections the results related to the 5-bus environment without opponent are not discussed in detail.

Rule-based agents. In this study, three entirely rule-based agents are considered (see Sec. 5.3). The do-nothing agent (dashed-grey in Fig. 4) represents a baseline agent which is completely inflexible. The greedy agent (dashed-purple in Fig. 4) and the Greedy-CAPA agent (dashed-brown in Fig. 4) represent benchmark approaches which include a high amount of simulation. Correspondingly, the do-nothing agent shows the worst performance of all considered approaches for all environments. For the 5-bus environment without opponent (with opponent) it survives about 10% (8.5% on average) of the possible timesteps, whereas for the 14-bus network it survives 18% (4.3% on average). In contrast, the two rule-based benchmark approaches show high performances for both the 5-bus environment with opponent (upper-right panel in Fig. 4) and the 14-bus environment without opponent (lower-left panel in Fig. 4). More precisely, in the 5-bus environment with opponent the Greedy-CAPA agent achieves 100% of the possible timesteps, whereas the greedy agent achieves 83% on average. In the 14-bus environment without opponent the order is reversed with the greedy agent achieving 96% of the possible timesteps, whereas the Greedy-CAPA agent achieves 85%. The 14-bus environment with opponent (lower-right panel in Fig. 4) is significantly more challenging than the previous environments. Consequently, the achieved timesteps of the greedy agent and the Greedy-CAPA agent are significantly smaller with only 7.7% and 7.4% on average, respectively.

Single RL baseline agent. The single RL agent (blue in Fig. 4) represents the most common RL-based baseline approach for L2RPN [27]. For the 5-bus environment with opponent (upper-right panel in Fig. 4) and the 14-bus environment without opponent (lower-left panel in Fig. 4), it exhibits performances similar to the rule-based benchmark approaches (i.e. the greedy and Greedy-CAPA agents). However, in the most challenging environment (14-bus with opponent, lower-right panel in Fig. 4) its performance drastically decreases and remains clearly below the already low performances of the two rule-based benchmark approaches.

Low-performing trained multi-agent systems. RL-CAPA (green in Fig. 4) and RL-Value Softmax (light-orange in Fig. 4) represent the two multi-agent architectures in which the regional agents are trained via RL, whereas the coordinating agent is rule-based (see Sec. 5.3). Both approaches show significantly worse performances than all other investigated agent architectures (except the do-nothing agent) for all environments (except the baseline 5-bus environment without opponent). In particular, the single RL baseline agent is outperforming RL-CAPA and RL-Value Softmax in both the 5-bus environment with opponent (upper-right panel in Fig. 4) and the 14-bus environment without opponent (lower-left panel in Fig. 4). Due to these very low performances we decided to not train these approaches in the significantly more challenging and computationally expensive 14-bus environment with opponent.

High-performing trained multi-agent systems. Greedy-RL (red in Fig. 4), RL-RL (orange in Fig. 4), and RL-Action Value RL (light-blue in Fig. 4) represent the multi-agent architectures in which the coordinating agent is trained via RL (see Sec. 5.2). The RL-RL multi-agent system and the RL-Action Value RL multi-agent system show very similar results such that we focus on RL-RL in the remainder of this section. All three approaches outperform the single RL baseline in the challenging 14-bus environments (lower panels in Fig. 4). The learning curves of RL-RL are initially below the learning curves of the single RL baseline but start to excel

[3]It should be noted that the training curves for the 14-bus network without an opponent have not yet fully converged. Due to the computational complexity involved in grid searching and generating all models across multiple seeds, training was halted once the curves had leveled off at 100,000 timesteps.

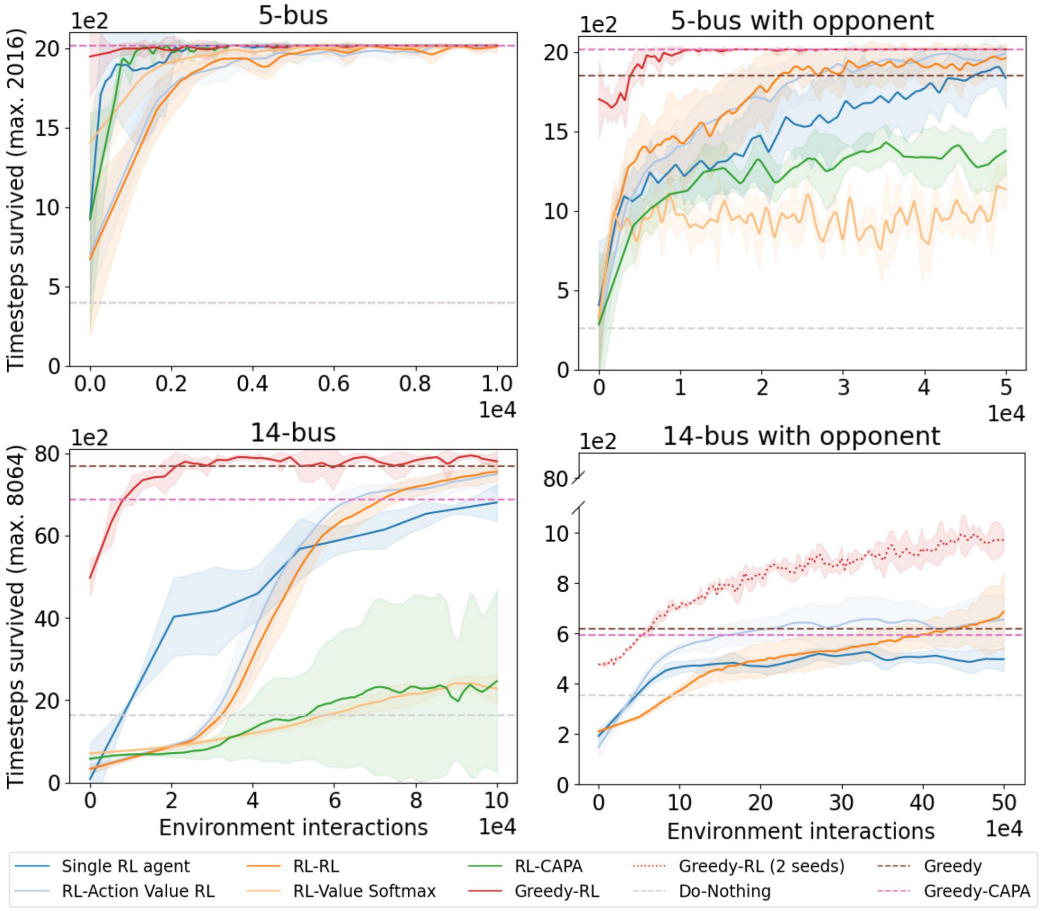


Figure 4: The mean number of timesteps survived on all validation scenarios throughout training for the 5-bus network without opponent (top left), the 5-bus network with opponent (top right), the 14-bus network without opponent (bottom left) and the 14-bus network with opponent (bottom right). For the mean and standard deviation, seeds are interpolated between 0 and `max_timesteps` to align them on a common time axis.

after a moderate amount of training. Importantly, RL-RL eventually also outperforms the rule-based benchmark approaches (i.e., the greedy agent and Greedy-CAPA) in the most challenging 14-bus environment with opponent (lower-right panel in Fig. 4).

The most impressive sample efficiency and performance is exhibited by Greedy-RL. In each of the four panels in Fig. 4, the training curve of Greedy-RL starts at a significantly higher level than those of the other trained agents. This is related to the employment of greedy agents as the regional agents such that the coordinating agent can only choose from valuable actions. Subsequently, Greedy-RL continues to exhibit the highest performance level during the entire training period in each environment. For the 5-bus environment with opponent (upper-right panel in Fig. 4) and the 14-bus environment without opponent (lower-left panel in Fig. 4), Greedy-RL converges quickly, effectively solving the environment while the training curves of the other trained agents only start to exhibit learning progress.

Also in the most challenging 14-bus environment with opponent (lower-right panel in Fig. 4), Greedy-RL exhibits extraordinary sample efficiency and performance. The learning curves of Greedy-RL start at the performance level of the converged single RL baseline models. We note that for Greedy-RL the learning curves related to different seeds, shown in Figure 5 in Appendix C, exhibit bimodal behavior (in contrast to all other architectures). That is, one group of seeds remains at a lower performance level (but still higher than the level of the converged single RL baseline) whereas a second group of seeds exhibits steep learning curves outperforming both the single RL baseline and the rule-based benchmark approaches (i.e., the greedy agent and Greedy-CAPA) by a large margin. Hence, a mean curve (averaged across all seeds) is not a good representation of the model behavior such that in the lower-right panel of Fig. 4 we only show an average across high-performing seeds. More details on the training curves related to individual seeds can be found in App. C.

For completeness, Table 9 in Appendix D shows model evaluation on the test scenarios, which exhibits results consistent with the evaluation on the validation scenarios during training. However, the focus of this study is on the *sample efficiency* of agent architectures, and, hence, in this study the model evaluation during training (Fig. 4) is more relevant than final test results (Tab. 9 in App. D).

7 Discussion

From the experiments it is evident that there can be benefits to factoring the action space. The Greedy-RL, RL-RL, and RL-Action Value RL agents consistently outperform the single RL agent, both asymptotically and in sample efficiency. With the larger grid, and especially with contingencies, this pattern becomes more evident.

There seems to be no significant difference in performance between the RL-RL and RL-Action Value RL agents. This suggests that the value of the regional agents' critics does not hold useful information for solving the coordination task.

On the other end, the Greedy-RL agent significantly outperforms RL-RL agent, both asymptotically and in sample efficiency. The improved sample efficiency could be due to the warm-start that the Greedy-RL agent can benefit from, as regional agents are already fixed. In contrast, agents like RL-RL may suffer from non-stationarity since both levels need to perform reasonably well before they can learn appropriately. Concretely, it can lead to poor performance when a regional agent selects a good action but a coordinator does not select the appropriate agent, or vice versa.

On the other hand, the Greedy-RL agent exhibits bi-modal behavior in the stochastic environment (Fig. 5 in App. C). One possible explanation is that not all contingency cases have equal value. It could be that only a smaller subset of the contingency cases pushes the agent to pick safer actions. This may have happened only on certain seeds. One possible explanation as to why we observe this only with Greedy-RL is its higher sample-efficiency. This would make the agent faster at picking up such patterns when (and if) they appear.

These results suggest that it is crucial to have long term reasoning in the coordination task. With an RL coordinator the Greedy-RL agent manages to outperform the Greedy baseline, especially in the challenging stochastic setting, despite the regional Greedy agents optimizing topological configurations for a single timestamp. The importance of long term reasoning in coordination is further supported by the low asymptotic performance of RL-CAPA and RL-Value Softmax. For the simplest setting, a 5-bus network without opponent, the coordinators such as CAPA-heuristic, random substation selection, Value Softmax still perform reasonably well. However, when used with added complexity due to an adversary or a larger network, these architectures no longer match the performance of a learned coordinator.

Moreover, when the network grows in size, the number of possible substations to select also grows. This makes the coordinator's task increasingly complex, calling the need for a more sophisticated coordinator.

Finally, because the Greedy-RL is among the best performing agents, it suggests that the factored action spaces can be tackled independently. In practice, the Greedy-RL agent is trained as a single RL agent within its coordination task. Thus, the regional agents can be trained independently of the coordinator. The coordinator can also be trained individually, but only after the regional agents. Most likely, this results in the significantly higher sample-efficiency of the Greedy-RL agent.

As a final note, Fig. 4 illustrates the stability of the proposed method by illustrating the standard deviation over ten different random seeds and multiple validation scenarios used during training. It stands out that some architectures exhibit considerably lower variability across validation scenarios and seeds, as reflected by smaller standard deviations, indicating more stable and robust convergence behavior. In contrast, other architectures show higher variability, suggesting that their performance is more sensitive to initialization.

8 Conclusion and Future Work

This study introduces a new approach that combines HRL with MARL to tackle the growing complexity of PNC in increasingly unpredictable environments. Our three-level architecture effectively breaks down the decision-making process, making control strategies more modular. One notable aspect is the introduction of a coordinating agent that considers both global state information and action proposals from regional agents.

8.1 Future Work

Future research could build on this study by exploring several avenues for extending and refining the proposed methods. Testing the scalability and robustness of the approach in larger networks, such as the 36-bus [26] and 118-bus networks [35], could provide insights into its effectiveness in more complex environments. To assess scalability, we conducted a preliminary evaluation on a 36-bus network (66,810 possible topological actions) and find promising outcomes. The results are shown in Appendix E. Moreover, future studies could test the impact of different strategies for decomposing the actions space [23].

Another promising direction is to investigate cooperative mechanisms among agents. While this study used a fully independent setup with PPO, sharing parameters or using a shared critic could potentially stabilize learning and enhance efficiency. Testing the impact of partial observability on regional agents could also provide more realistic insights into local versus global information.

The results also suggest more concrete next steps for making the CCMA more scalable. The Greedy-RL architecture shows the most promise in sample efficiency. However, it is also the most computationally expensive agent as detailed in Appendix F. To mediate this, one could use imitation learning to train regional agents. These agents would try to imitate the greedy agent directly just much faster.

Furthermore, the regional agents can be trained independently of the coordinator using RL, similar to a single-agent RL setting where the action space is confined to a single regional agent. This approach allows regional agents to incorporate some level of longer-term planning, albeit with a shorter time horizon compared to the coordinator that could be limited to a few timesteps. The aforementioned imitation learning can be used as warm start in this setting.

However, it should be noted that our approach does not factorize the observation space. With the current observation space, when training the regional agents separately, the data should include a lot of varied configurations of the observed section of the grid. This applies also when using only imitation learning. Each regional agent observes the entire grid, and thus all other regions. Therefore the amount of data needed to train an individual agents scales exponentially with the number of regions.

One could try to give partial observations to the regional agents. Each agent only taking as observation a neighborhood of its region (included). Thus, the data to train a regional agents should only include combinations within its neighborhood. This effectively creates a cap on the amount of data needed to train an agent, with respect to the number of regions. In such a setting, the amount of data scales with respect to the size of the neighborhood and not the total number of regions. Expanding the grid outside of a regional agent's neighborhood will not affect the amount of data needed to train it. The connections going out of the neighboring regions can still be kept and simulated. The data can include various level of loading on these connections. Effectively the part outside of the neighborhood can be seen as grid injections. It should be noted that these observation spaces would not be completely independent as there would be overlap between them.

Acknowledgments

We want to thank Erica van der Sar and Alessandro Zocca from the Vrije Universiteit Amsterdam for their valuable insights while conducting the research as well as feedback in refining this work.

AI4REALNET has received funding from European Union's Horizon Europe Research and Innovation programme under the Grant Agreement No 101119527. Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

Davide Grossi acknowledges support by the Hybrid Intelligence Center, a 10-year program funded by the Dutch Ministry of Education, Culture and Science through the Netherlands Organisation for Scientific Research (NWO).

References

[1] Melike Baykal-Gursoy. 2010. Semi-Markov Decision Processes. *Wiley Encyclopedia of Operations Research and Management Science* (2010). doi:10.1002/9780470400531.eorms0757

[2] Anandsingh Chauhan, Mayank Baranwal, and Ansuma Basumatary. 2023. Powrl: A reinforcement learning framework for robust management of power networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 14757–14764.

[3] Peter Dayan and Geoffrey E Hinton. 1992. Feudal reinforcement learning. *Advances in neural information processing systems* 5 (1992).

[4] Matthijs de Jong, Jan Viebahn, and Yuliya Shapovalova. 2024. Imitation Learning for Intra-Day Power Grid Operation through Topology Actions. *arXiv preprint arXiv:2407.19865* (2024).

[5] Benjamin Donnot. 2019. Source code for grid2op.Opponent.randomLineOpponent. https://grid2op.readthedocs.io/en/dev_multiagent/_modules/grid2op/Opponent/randomLineOpponent.html.

[6] Benjamin Donnot. 2020. Grid2op- A testbed platform to model sequential decision making in power systems. https://GitHub.com/rte-france/grid2op.

[7] Benjamin Donnot. 2020. L2RPN Baselines- Repository hosting reference baselines for the L2RPN challenge. https://github.com/Grid2op/l2rpn-baselines/.

[8] Matthias Dorfer, Anton R Fuxjäger, Kristian Kozak, Patrick M Blies, and Marcel Wasserer. 2022. Power grid congestion management via topology optimization with AlphaZero. *arXiv preprint arXiv:2211.05612* (2022).

[9] Shixiang Gu, Ethan Holly, Timothy Lillicrap, and Sergey Levine. 2017. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 3389–3396.

[10] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*. PMLR, 1861–1870.

[11] Majid Heidarifar, Panagiotis Andrianesis, Pablo Ruiz, Michael C Caramanis, and Ioannis Ch Paschalidis. 2021. An optimal transmission line switching and bus splitting heuristic incorporating AC and N-1 contingency constraints. *International Journal of Electrical Power & Energy Systems* 133 (2021), 107278.

[12] Alex Kendall, Jeffrey Hawke, David Janz, Przemyslaw Mazur, Daniele Reda, John-Mark Allen, Vinh-Dieu Lam, Alex Bewley, and Amar Shah. 2019. Learning to drive in a day. In *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 8248–8254.

[13] Hans-Jurgen Koglin and Holger Müller. 1982. Corrective switching: a new dimension in optimal load flow. *International Journal of Electrical Power & Energy Systems* 4, 2 (1982), 142–149.

[14] Tu Lan, Jiajun Duan, Bei Zhang, Di Shi, Zhiwei Wang, Ruisheng Diao, and Xiaohu Zhang. 2020. AI-based autonomous line flow control via topology adjustment for maximizing time-series ATCs. In *2020 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 1–5.

[15] Malte Lehna, Clara Holzhüter, Sven Tomforde, and Christoph Scholz. 2024. Hugo–highlighting unseen grid options: Combining deep reinforcement learning with a heuristic target topology approach. *Sustainable Energy, Grids and Networks* 39 (2024), 101510.

[16] Malte Lehna, Jan Viebahn, Antoine Marot, Sven Tomforde, and Christoph Scholz. 2023. Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents. *Energy and AI* 14 (2023), 100276.

[17] Yuanlong Li, Yonggang Wen, Dacheng Tao, and Kyle Guan. 2019. Transforming cooling optimization for green data center via deep reinforcement learning. *IEEE transactions on cybernetics* 50, 5 (2019), 2002–2013.

[18] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Joseph Gonzalez, Ken Goldberg, and Ion Stoica. 2017. Ray rllib: A composable and scalable reinforcement learning library. *arXiv preprint arXiv:1712.09381* 85 (2017).

[19] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez, and Ion Stoica. 2018. Tune: A Research Platform for Distributed Model Selection and Training. *arXiv preprint arXiv:1807.05118* (2018).

[20] Emily Little, Sandrine Bortolotti, Jean-Yves Bourmaud, Efthymios Karangelos, and Yannick Perez. 2021. Optimal transmission topology for facilitating the growth of renewable power generation. In *2021 IEEE Madrid PowerTech*. IEEE, 1–6.

[21] Michael L Littman. 1994. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*. Elsevier, 157–163.

[22] Shunyu Liu, Yanzhen Zhou, Mingli Song, Guangquan Bu, Jianbo Guo, and Chun Chen. 2024. Progressive decision-making framework for power system topology control. *Expert Systems with Applications* 235 (2024), 121070.

[23] Gianvito Losapio, Davide Beretta, Marco Mussi, Alberto Maria Metelli, and Marcello Restelli. 2024. State and Action Factorization in Power Grids. arXiv:2409.04467 [eess.SY] https://arxiv.org/abs/2409.04467

[24] Trieu T Mai, Paige Jadun, Jeffrey S Logan, Colin A McMillan, Matteo Muratori, Daniel C Steinberg, Laura J Vimmerstedt, Benjamin Haley, Ryan Jones, and Brent Nelson. 2018. Electrification futures study: scenarios of electric technology adoption and power consumption for the United States.

[25] Blazej Manczak, Jan Viebahn, and Herke van Hoof. 2023. Hierarchical Reinforcement Learning for Power Network Topology Control. https://arxiv.org/abs/2311.02129.

[26] Antoine Marot, Benjamin Donnot, Karim Chaouache, Adrian Kelly, Qiuhua Huang, Ramij-Raja Hossain, and Jochen L Cremer. 2022. Learning to run a power network with trust. *Electric Power Systems Research* 212 (2022), 108487.

[27] Antoine Marot, Benjamin Donnot, Gabriel Dulac-Arnold, Adrian Kelly, Aidan O'Sullivan, Jan Viebahn, Mariette Awad, Isabelle Guyon, Patrick Panciatici, and Camilo Romero. 2021. Learning to run a power network challenge: a retrospective analysis. In *NeurIPS 2020 Competition and Demonstration Track*. PMLR, 112–132.

[28] Antoine Marot, Benjamin Donnot, Sami Tazi, and Patrick Panciatici. 2018. Expert system for topological remedial action discovery in smart grids. In *Mediterranean Conference on Power Generation, Transmission, Distribution and Energy Conversion (MEDPOWER 2018)*. IET, 1–6.

[29] Ian Osband and Benjamin Van Roy. 2014. Near-optimal reinforcement learning in factored mdps. *Advances in Neural Information Processing Systems* 27 (2014).

[30] Patrick Panciatici, Gabriel Bareux, and Louis Wehenkel. 2012. Operating in the fog: Security management under uncertainty. *IEEE Power and Energy Magazine* 10, 5 (2012), 40–49.

[31] polixer. 2020. Winner of L2RPN ICAPS 2021. https://github.com/polixir/L2RPN_2021.

[32] Pablo Ariel Ruiz, Evgeniy Goldis, Aleksandr M Rudkevich, Michael C Caramanis, C Russ Philbrick, and Justin M Foster. 2016. Security-constrained transmission topology control MILP formulation using sensitivity factors. *IEEE Transactions on Power Systems* 32, 2 (2016), 1597–1605.
[33] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. 2020. Mastering atari, go, chess and shogi by planning with a learned model. *Nature* 588, 7839 (2020), 604–609.
[34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
[35] Gaëtan Serré, Eva Boguslawski, Benjamin Donnot, Adrien Pavão, Isabelle Guyon, and Antoine Marot. 2022. Reinforcement learning for Energies of the future and carbon neutrality: a Challenge Design. *arXiv preprint arXiv:2207.10330* (2022).
[36] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
[37] Richard S Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112, 1 (1999), 181–211. doi:10.1016/S0004-3702(99)00052-1
[38] Erica van der Sar, Alessandro Zocca, and Sandjai Bhulai. 2023. Multi-Agent Reinforcement Learning for Power Grid Topology Optimization. arXiv:2310.02605 [cs.LG] https://arxiv.org/abs/2310.02605
[39] Jan Viebahn, Sjoerd Kop, Joost van Dijk, Hariadi Budaya, Marja Streefland, Davide Barbieri, Paul Champion, Mario Jothy, Vincent Renault, and Simon Tindemans. 2024. GridOptions Tool: Real-World Day-Ahead Congestion Management using Topological Remedial Actions. https://cse.cigre.org/cse-n035/c2-gridoptions-tool-real-world-day-ahead-congestion-management-using-topological-remedial-actions.html. *CIGRE Session 2024* (2024).
[40] Jan Viebahn, Matija Naglic, Antoine Marot, Benjamin Donnot, and Simon H Tindemans. 2022. Potential and challenges of AI-powered decision support for short-term system operations. *CIGRE Session 2022* (2022).
[41] Deunsol Yoon, Sunghoon Hong, Byung-Jun Lee, and Kee-Eung Kim. 2020. Winning the l2rpn challenge: Power grid management via semi-markov afterstate actor-critic. In *International Conference on Learning Representations*.
[42] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. 2022. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems* 35 (2022), 24611–24624.

A Comparative Analysis of Power Grid Control Architectures

Table 4 compares the proposed CCMA architecture and other state-of-the-art architectures, focusing on agent architecture, action space representation and action space reduction.

B Hyperparameters

A grid search was performed for all architectures in all different set-ups, of which the explored values can be found here. Unless otherwise mentioned, the default parameters of Ray's PPO were used [18].

B.1 5-bus network without opponent

Grid search was performed over the following parameters:

- Learning rate (lr): 0.01, 0.001, 0.0001, 0.00001
- SGD minibatch size (mbs): 32, 64
- Train batch size (bs): 64, 128

Depending on the result of the grid search, the learning rate parameter was further explored to include half and five times its best setting. For example, if 0.001 was the best learning rate, the next grid search also explored 0.005 and 0.0005. Additionally, the number of SGD iterations (it) was fixed at 15. Table 5 illustrates all chosen parameters.

Table 5: Chosen parameters for all learning architectures on the 5-bus network without opponent.

Agent	lr	mbs	bs	it
Single RL	0.01	32	64	15
RL-RL	0.0005	64	128	15
RL-Action Value RL	0.0005	64	128	15
RL-Action Softmax	0.0005	64	128	15
RL-CAPA	0.005	32	64	15
RL-Random	0.01	64	64	15
Greedy-RL	0.0005	32	128	15

B.2 5-bus network with opponent

Grid search was performed over the following parameters:

- Learning rate (lr): 0.001, 0.0001, 0.00001, 0.000001
- SGD minibatch size (mbs): 32, 64, 128, 256
- Train batch size (bs): 128, 256, 512, 1024
- Number of SGD iterations (it): 5, 10

Depending on the result of the grid search, the learning rate parameter was further explored to include nearby values. For example, if 0.001 was the best learning rate, the next grid search also explored 0.0005, 0.0007, 0.003, 0.005. Table 6 illustrates all chosen parameters.

Table 6: Chosen parameters for all learning architectures on the 5-bus network with opponent.

Agent	lr	mbs	bs	it
Single RL	0.0007	128	512	10
RL-RL	0.0007	64	128	5
RL-Action Value RL	0.0003	128	256	5
RL-Value Softmax	0.0005	64	128	10
RL-CAPA	0.0005	128	256	5
RL-Random	0.0001	64	128	5
Greedy-RL	0.00005	64	256	10

B.3 14-bus network without opponent

Grid search for the 14-bus networks was performed in two batches. The first batch was a coarse grid search over the following settings:

- Learning rate (lr): 0.0001, 0.00001, 0.000001
- SGD minibatch size (mbs): 64, 256, 1024
- Train batch size (bs): 128, 512, 2048
- Number of SGD iterations (it): 5, 10, 15

The second batch further investigated the learning rate, SGD minibatch size and train batch size parameters. For example, if 0.001 was the best learning rate, the next grid search also explored 0.0005, 0.0007, 0.003, 0.005. For the batch sizes, the surrounding integers x in 2^x were explored. Additionally, new parameters were explored:

- Value function clip parameter (vfc): 10, 20, 50
- Clip parameter (cp): 0.15, 0.2, 0.3
- Value function loss coefficient (vfl): 0.9, 0.95, 1
- Lambda (λ): 0.9, 0.92, 0.95, 0.98, 1

Table 7 illustrates all chosen parameters.

Table 4: Comparison of RL-based Architectures for Power Grid Topology Control.

Paper	Agent Architecture	Type of Action Space for RL Agent	Action Space Reduction
CCMA, proposed in this study	Hierarchical multi-agent architecture that separates decision layers between, first, regional agents and, second, a coordinator. Distributed proposals over multiple regional agents yield a diverse range of candidate actions. Directly addresses the combinatorial challenge by decomposing the action space, so the problem scales with the sum of sub-problem sizes rather than their product.	RL coordinator learns on higher-level action space consisting of the set of controllable substations. This action space is naturally much smaller than the primitive action space.	No arbitrary action space reduction is necessary. Only power system principles are used to reduce the action space, without relying on heuristics or manual tuning. This also avoids introducing bias through greedy action selection.
Liu et al. [22]	Hierarchical agent architecture similar to Manczak et al. [25], as discussed in Sec. 5.2 in this study. That is, first a coordinator specifies a substation, and subsequently a single regional agent specifies the substation configuration.	As in CCMA, the coordinator learns on higher-level action space consisting of the set of controllable substations. However, the coordinator does not receive the output of the regional agent as input, as discussed in Sec. 5.2 in this study.	Similar to CCMA, that is, no action space reduction but action space decomposition. However, no action space decomposition on the subsation level.
Yoon et al. [41]	Hierarchical approach consisting of, first, a RL agent choosing a goal topology, and, second, a rule-based agent to specify the corresponding sequence of primitive substation changes. Moreover, an afterstate representation is used.	RL agent learns on entire topology action space which, hence, needs to be drastically reduced.	The action space consists of goal topologies, which suffers even more from combinatorial explosion than substation reconfiguration actions. How the action space is reduced is not discussed in the paper, but the code includes an arbitrary cut-off range of maximum possible actions per substation.
Lehna et al. [16]	Employs a teacher-tutor-junior-senior framework. Essentially, this is a single RL approach with a warm-start based on greedy expert data.	RL agent needs to learn on the primitive action space which, hence, needs to be drastically reduced.	Uses brute-force greedy search to create a frequency distribution of the entire action space. Subsequently, the action space is then drastically pruned based on frequency distribution using an arbitrary cut-point.
Lehna et al. [15]	Extending Lehna et al. [16] by adding a rule-based application of so-called target topologies in a low loading regime.	Same as Lehna et al. [16], that is, RL agent needs to learn on the primitive action space which, hence, needs to be drastically reduced.	Similar to Lehna et al. [16]. Reduced set of target topologies is based greedy agent statistics. The reduced set of primitive actions is simply taken from another agent by Dorfer et al. [8].
Chauhan et al. [2]	Essentially a single RL approach supported by a number of domain specific heuristics similar to Lehna et al. [16], see also Sec. 4.1.1 in this study.	Same as Lehna et al. [16], that is, RL agent needs to learn on the primitive action space which, hence, needs to be drastically reduced.	Similar to Lehna et al. [16]. Through extensive brute-force simulation, the action space is drastically reduced; the actual action space reduction procedure is not explained.
Dorfer et al. [8]	Essentially a single RL approach using Monte Carlo Tree Search.	Same as Lehna et al. [16], that is, RL agent needs to learn on the primitive action space which, hence, needs to be drastically reduced.	Similar to Lehna et al. [16]. Through extensive brute-force simulation, the action space is drastically reduced by using the top most frequent actions.

Table 7: Chosen parameters for all learning architectures on the 14-bus network without opponent.

Agent	lr	mbs	bs	it	cp	vfc	vfl	λ
Single RL	0.00005	256	1024	15	0.3	10	1	0.95
RL-RL	0.00007	512	256	10	0.15	10	1	0.95
RL-Action Value RL	0.0001	256	512	10	0.15	20	0.95	0.95
RL-Value Softmax	0.00005	64	256	10	0.15	10	1	0.92
RL-Capa	0.00005	128	256	10	0.3	10	1	1
RL-Random	0.00001	256	256	10	0.3	10	1	0.92
Greedy-RL	0.0005	128	512	15	0.3	10	0.95	0.95

B.4 14-bus network with opponent

Grid search for the 14-bus networks was performed in two batches. The first batch was a coarse grid search over the following settings:

- Learning rate (lr): 0.0001, 0.00001, 0.000001
- SGD minibatch size (mbs): 64, 256, 1024
- Train batch size (bs): 128, 512, 2048
- Number of SGD iterations (it): 5, 10, 15

The second batch further investigated the learning rate, SGD minibatch size and train batch size parameters. For example, if 0.001 was the best learning rate, the next grid search also explored 0.0005, 0.0007, 0.003, 0.005. For the batch sizes, the surrounding integers x in 2^x were explored. Additionally, new parameters were explored:

- Value function clip parameter (vfc): 10, 20, 50
- Clip parameter (cp): 0.15, 0.2, 0.3
- Value function loss coefficient (vfl): 0.9, 0.95, 1
- Lambda (λ): 0.9, 0.92, 0.95, 0.98, 1

Table 8 illustrates all chosen parameters.

C Individual Seeds for 14-Bus Network With Opponent

Figure 5 shows the mean performance of the Single RL agent and compares it to the individual seeds of the RL-RL agent (left) and the Greedy-RL agent (right) on the 14-bus environment with opponent. The training of the Greedy-RL agent is very computationally

Table 8: Chosen parameters for all learning architectures on the 14-bus network with opponent.

Agent	lr	mbs	bs	it	cp	vfc	vfl	λ
Single RL	0.0005	512	2048	5	0.15	10	0.95	0.95
RL-RL	0.0001	1024	1024	10	0.15	10	0.95	0.95
RL-Action Value RL	0.0001	1024	2048	10	0.15	10	0.95	0.95
Greedy-RL	0.0001	1024	1024	10	0.2	10	1	0.95

expensive due to the simulation of the entire action space and the stochastic nature of the environment (Sec. 4.2). We were able to complete training for the 14-bus environment with opponent only on two of the ten seeds. However, we still observe that it consistently outperforms the average performance of the single RL agent, both asymptotically and in sample efficiency.

D Results of test scenarios

Table 9 shows the number of time steps survived in the test scenarios. For each architecture, the results of best-performing model seed in the validation scenarios is shown.

E Preliminary 36-bus Experiment

To explore scalability on the 36-bus network (*l2rpn_icaps_2021_large*), we experimented with the Greedy, Single RL and Greedy-RL agents.

First, we deployed a more efficient greedy agent that evaluates all non-hub substations (fewer than 1,000 actions) and executes the first move whose simulated peak line load remains below the safety threshold or decreases by at least 5%, thus avoiding full combinatorial search over hub actions. Only if no non-hub action qualifies, the agent continuously samples from the 65,121 hub actions until a good action is found. If still none meet the criteria, it selects the action minimizing the maximum line loading.

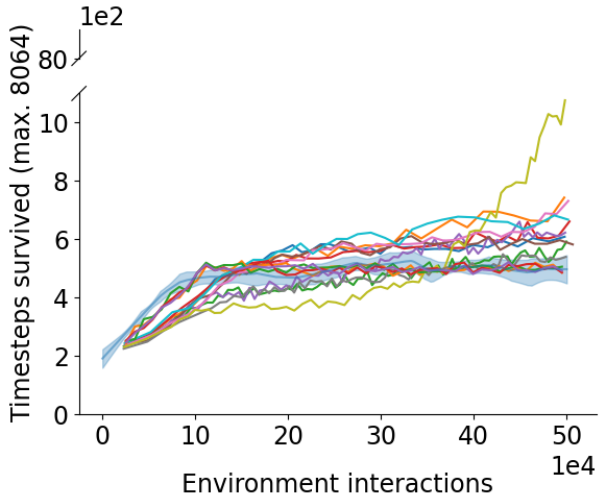
Next, we built a reduced action space by collecting every hub action chosen more than once by this greedy agent (while retaining all non-hub actions). We used this action space to train four and two sets of hyperparameters of both the Greedy-RL and Single-RL agents, respectively. Based on App. B, we evaluated the parameter setting specified in Table 10.

Figure 6 shows the resulting training (a) and validation (b) curves. The results show that both the Single RL and Greedy-RL suffer from overfitting, suggesting that further hyperparameter tuning is needed. While the training curves consistently increase (see Fig. 6a), the validation curves show large variations (see Fig. 6b). It should be noted that when an agent is updated, its buffer will contain experiences from only a fraction of the training scenarios. One likely explanation for this overfitting (and the subsequent recovery) is that out of distribution training scenarios are picked for training (at around 200k environmental interactions in Fig. 6b). This most likely results in the agent trying to perform well in these peculiar scenarios, effectively "forgetting" some of the behaviors useful in the validation set.

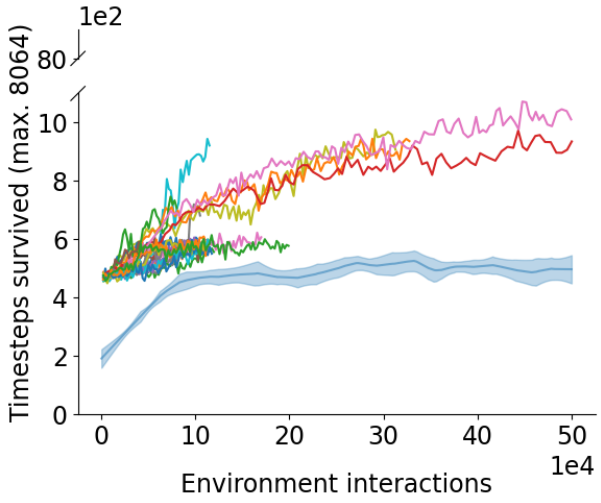
Regardless, the plots still highlight the value of Greedy-RL in terms of performance. The best validation performance achieved by Greedy-RL far surpasses that of both the Single-RL and the rule-based Greedy agent. These findings confirm our smaller-grid conclusions while underscoring the need for a full hyperparameter grid search. Future work will also explore splitting the hub agent into multiple regional agents, allowing the full range of actions to be used again.

F Computational Time

Table 11 presents the average computational time per environment interaction during training for various agents on the 14-bus network with an opponent, using a single CPU per worker. The Single RL agent demonstrates the lowest computational time, followed closely by the RL-RL and RL-Action Value RL agents. In contrast, the Greedy-RL agent exhibits significantly higher computational time, approximately an order of magnitude greater than the others.

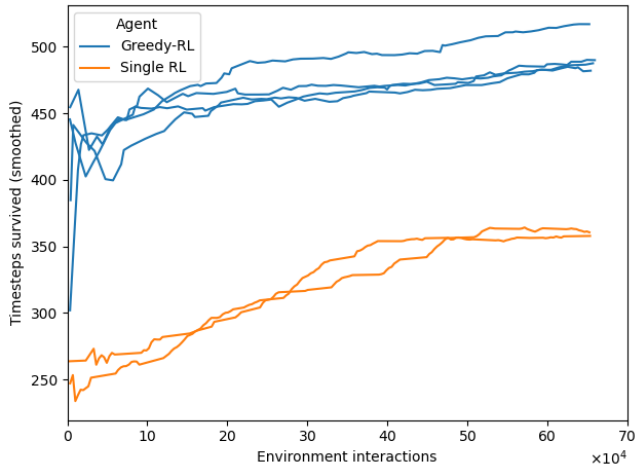


(a) The individual seeds for the RL-RL Agent compared to the mean and standard deviation of the Single RL Agent (blue).

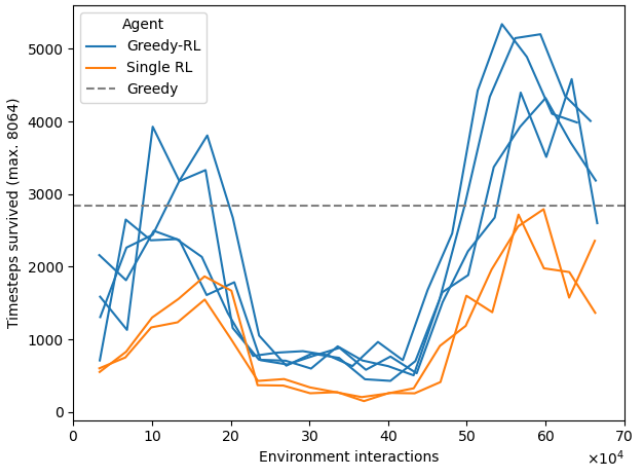


(b) The individual seeds for the Greedy-RL Agent compared to the mean and standard deviation of the Single RL Agent (blue).

Figure 5: The mean performance of the Single RL Agent compared to the individual seeds of the Greedy-RL and RL-RL agent on the 14-bus environment with opponent. For the mean and standard deviation, seeds are interpolated between 0 and `max_timesteps` to align them on a common time axis.



(a) The rolling mean (window = 50) number of timesteps survived on training scenarios for the 36-bus network without opponent. As described in Sec. 4.1.1, the agent is, whenever feasible, trained on two-day segments (576 timesteps) extracted from the original training scenarios. When nighttime instabilities prevent a full two-day run, these segments may occasionally be one day shorter or longer. Across all training scenarios, the average segment length is 588.1 timesteps.



(b) The mean number of timesteps survived on all 296 validation scenarios throughout training for the 36-bus network without opponent.

Figure 6: Performance of the Greedy, Greedy-RL and Single RL agents on the 36-bus network without opponent over time: (a) training, (b) validation.

Table 9: Performance of different agents (best performing model seed on validation scenarios) in the test scenarios of the 5-bus and 14-bus networks, with and without opponent, in terms number of timesteps survived. Shown are the mean and standard deviation (sd) across different scenarios (and different environment seeds in case of opponent).

Architecture	# ts survived (sd) in 5-bus network		# ts survived (sd) in 14-bus network	
	Regular	Opponent	Regular	Opponent
Do-Nothing	212.8 (208.5)	172.0 (187.6)	1527.9 (1343.3)	349.5 (326.7)
Single RL	2016.0 (0.0)	2016.0 (0.0)	6744.4 (2289.7)	516.8 (468.4)
RL-CAPA	2016.0 (0.0)	1514.3 (747.1)	1533.6 (1353.8)	-
Greedy	2016.0 (0.0)	1662.3 (664.8)	7716.8 (1305.6)	622.4 (500.0)
Greedy-CAPA	2016.0 (0.0)	2016.0 (0.0)	6841.8 (2109.3)	593.6 (434.5)
Greedy-RL	2016.0 (0.0)	2016.0 (0.0)	7853.8 (952.0)	923.7 (893.5)
RL-Value Softmax	2016.0 (0.0)	1270.2 (799.5)	2316.0 (1987.9)	-
RL-RL	2016.0 (0.0)	1977.6 (242.5)	7510.7 (1600.5)	1122.4 (1033.6)
RL-Action Value RL	2016.0 (0.0)	1976.7 (248.7)	7410.0 (1531.0)	835.6 (764.7)

Table 10: Hyperparameter configurations for Greedy-RL and Single-RL on the 36-bus network without opponent.

Agent	lr	mbs	bs	it	cp	vfc	vfl	λ
Greedy-RL	0.0001	512	3072	10	0.1	20	0.95	0.95
Greedy-RL	0.00001	512	3072	10	0.1	20	0.95	0.95
Greedy-RL	0.00005	1024	3072	5	0.2	20	0.95	0.95
Greedy-RL	0.00005	1024	3072	5	0.2	20	0.95	0.95
Single-RL	0.00005	512	3072	10	0.1	20	0.95	0.95
Single-RL	0.00005	512	3072	10	0.1	20	0.95	0.95

Table 11: Compute Time Per Timestep While Training 14-Bus Network With Opponent

Agent	Time per Environment Interaction (s)
Single RL	0.307
RL-RL	0.492
RL-Action Value RL	0.468
Greedy-RL	5.089